

Notes on LangChain Components

Quick Recap of Previous

- **LangChain = Open-source framework** → helps build LLM-powered apps.
 - Why needed? → Building apps like “*Chat with a PDF*” from scratch is hard.
 - LangChain helps by:
 - Orchestrating components efficiently.
 - Creating **chains** (pipelines).
 - Being **model-agnostic** (easy to switch between OpenAI, Google, etc.).
 - Example Use Cases:
 - Chatbots
 - Knowledge Assistants
 - AI Agents
-

6 Core Components of LangChain

1. **Models**
 2. **Prompts**
 3. **Chains**
 4. **Memory**
 5. **Indexes**
 6. **Agents**
-

1 Models

- **Definition:** Core interface to interact with AI models.
 - **Why needed?**
 - Different LLM providers (OpenAI, Anthropic, etc.) have different APIs → messy.
 - LangChain **standardizes** this, so switching providers = just 1–2 lines of code.
 - **Types:**
 - **Language Models (LLMs):** Input = text → Output = text. (e.g., Chatbots, Agents).
 - **Embedding Models:** Input = text → Output = vector (used for semantic search, similarity).
 - **Supported Providers:** OpenAI, Anthropic, Azure, HuggingFace, Google, AWS, etc.
-

2 Prompts

- **Definition:** Input you give to an LLM.
- **Importance:** Output depends heavily on prompt wording.
- **Prompt Engineering = new career field!**
- **Types of Prompts in LangChain:**

1. **Dynamic & Reusable Prompts:** Use placeholders (topic, tone, etc.) and fill dynamically.
 2. **Role-based Prompts:** Define system roles (e.g., “You are an experienced doctor”).
 3. **Few-shot Prompts:** Give examples first, then ask the actual query.
-

3 Chains

- **Definition:** Pipeline of tasks in LangChain.
 - **Why useful?**
 - Automatically passes one stage’s output → next stage’s input.
 - No manual coding for data transfer.
 - **Examples:**
 1. **Sequential Chain:** Translate English → Hindi, then summarize in Hindi.
 2. **Parallel Chain:** Multiple LLMs work on the same input, then results combined.
 3. **Conditional Chain:** Based on condition → take different action (e.g., positive vs negative feedback).
-

4 Indexes

- **Definition:** Connects LLMs to **external knowledge** (PDFs, websites, databases).
 - **4 Parts of Indexes:**
 1. **Document Loader:** Load data (PDF, DB, etc.).
 2. **Text Splitter:** Break into chunks.
 3. **Vector Store:** Store embeddings (special database).
 4. **Retriever:** Fetch relevant data when user queries.
 - **Use Case Example:**
 - Chatbot for company policy.
 - LLM alone can’t answer (not trained on private data).
 - With **indexes**, it can search company documents and reply.
-

5 Memory

- **Problem:** LLM API calls are **stateless** (forget previous conversation).
 - **Solution = Memory:** Adds context/history to LLM apps.
 - **Types of Memory in LangChain:**
 1. **Conversation Buffer Memory:** Store all chat history.
 2. **Conversation Buffer Window Memory:** Store last N interactions.
 3. **Summary Memory:** Store a summary instead of entire history.
 4. **Custom Memory:** Store specific facts/preferences.
-

6 Agents

- **Difference from Chatbots:**
 - Chatbot = only conversation.
 - Agent = can also **take actions** (API calls, tool use).

- **How it works:**
 - Agent has **reasoning ability** + **tool access**.
 - Example:
 - Query: “*Multiply today’s Delhi temperature by 3.*”
 - Agent:
 - Uses **weather API** → gets Delhi temp = 25°C.
 - Uses **calculator tool** → $25 \times 3 = 75$.
 - Returns **75** as answer.
 - **In short:** Agent = Chatbot + Superpowers 🚀
-

🎯 Conclusion

- 6 components = backbone of LangChain: **Models, Prompts, Chains, Memory, Indexes, Agents**.
 - Future videos → deep dive into each component (starting with Models).
-

💡 Interactive Tip for Revision:

- 👉 Try recalling all 6 components in order without looking.
- 👉 Can you explain “Agents” to a friend in one sentence?